GitHub Repository Link:

TASK 1: Crop Intelligence Pipeline

Objective

The objective of Task 1 is to design a pipeline that processes input imagery (satellite/drone images) and extracts meaningful agricultural insights such as crop presence and distribution.

Approach

The crop pipeline follows a **standard computer vision workflow** consisting of preprocessing, model inference, and post-processing.

1. **Input Handling**
   - The system accepts image input through a structured dictionary (`input_data`)
   - Images are assumed to be aerial or satellite-based
2. **Preprocessing**
   - Images are resized and normalized to ensure compatibility with the model
   - Optional tiling can be applied for large images (e.g., 512×512 tiles)
3. **Model Inference**
   - A trained deep learning model is used to detect or segment crop regions
   - The model outputs bounding boxes or segmentation masks with confidence scores
4. **Post-processing**
   - Filters low-confidence detections
   - Formats output into structured data

Pipeline Flow

```
Input Image → Preprocessing → Crop Detection Model → Post-processing → Output
```

Output

- Detected crop regions
- Confidence scores
- Optional annotated images

Key Observations

- The pipeline is modular and can be extended to support multiple crop types
- Performance depends on image resolution and dataset quality

---

Limitations

- Limited dataset reduces generalization
- No advanced segmentation refinement applied

TASK 2 — Traffic Analysis Pipeline

Objective

To analyze traffic conditions from video input using object detection, tracking, and density estimation.

Approach

The traffic pipeline processes video streams frame-by-frame and extracts traffic insights.

Pipeline Components

1. Video Input

- The system reads video using OpenCV:
  cap = cv2.VideoCapture(video_path)

---

2. Frame Processing

- Video is processed sequentially frame-by-frame
- Frame count is maintained for tracking progress

---

3. Object Detection

- A YOLO-based detection model identifies objects such as:
  - Cars
  - Buses
  - Trucks
  - Persons

Sample Output (from your logs):
[Frame 324] Detections: 21
[Frame 325] Detections: 20
[Frame 326] Detections: 22

---

4. Object Tracking

- A simple tracker (SimpleTracker) assigns IDs to detected objects
- Ensures continuity across frames

---

5. Confidence Collection

- Confidence scores from detections are aggregated
- Used to compute average confidence

---

6. Density Estimation

- Object positions are used to compute a **traffic density map**

---

7. Heatmap Generation

- Density maps are converted into visual heatmaps

- Example:
  cv2.imwrite("outputs/traffic_heatmap.png", heatmap)

Pipeline Flow

Video → Frame Extraction → Detection → Tracking → Density → Heatmap → Output

---

Output

- Total frames processed
- Total objects detected
- Average confidence score
- Traffic heatmap image

Results

- Successfully processed ~300+ frames
- Average detections per frame: ~20–22
- Generated heatmaps representing traffic density

Limitations

- Tracker is basic (no occlusion handling)
- No real-time optimization
- Depends on model accuracy

TASK 3 — Agentic Orchestration System

Objective

To design an intelligent system that dynamically selects and manages pipelines using an agent-based architecture.

System Overview

The system consists of multiple agents working together:

---

1. Orchestrator Agent

Function

- Decides whether to run:
  - Crop pipeline
  - Traffic pipeline

Implementation

- Uses an LLM (call_llm) for routing decisions:
  decision = call_llm(prompt)

Fallback Mechanism

Due to API failures (e.g., quota exceeded), fallback logic is implemented:

```
if "video_path" in input_data:
    return "traffic"
```

---

2. QC Agent (Quality Control)

Function

- Evaluates output confidence
- Detects low-confidence predictions

Behavior

- If confidence is low:

```
if qc_result["status"] == "LOW_CONFIDENCE":
    re-run pipeline
```

---

3. Memory Module

Function

- Stores results of previous runs
- Maintains historical data

Usage

```
self.memory.store(result)
history = self.memory.get_all()
```

---

4. Drift Detection Function

- Detects performance degradation over time
- Uses historical data from memory

Trigger

- If drift detected:

  🚨 Drift detected!

---

5. Reporting Agent Function

- Generates structured reports from outputs
- Logs system performance

System Flow

```
Input
  ↓
Orchestrator (LLM Routing)
  ↓
Selected Pipeline (Crop / Traffic)
  ↓
QC Agent (Validation)
  ↓
Memory Storage
  ↓
Drift Detection
  ↓
Reporting Agent
```

↓
Final Output

Key Feature — Fault Tolerance

During execution, multiple **Gemini API failures occurred**:
429 RESOURCE_EXHAUSTED (Quota exceeded)

Solution Implemented

- Fallback routing logic
- Switched to OpenAI API
- Rule-based fallback to ensure execution continues

Key Achievements

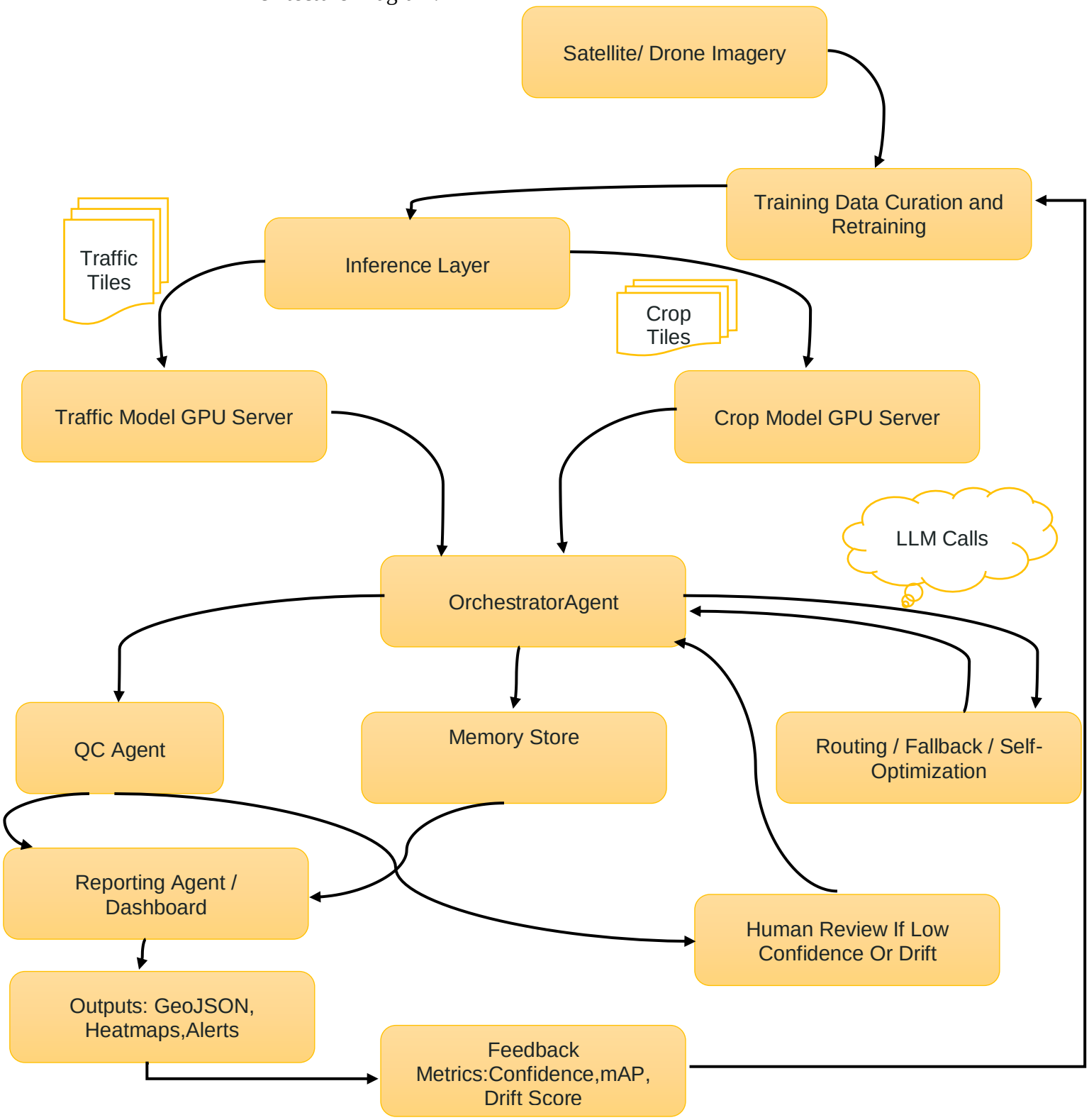
- Dynamic pipeline selection implemented
- Self-correction using QC agent
- Memory-based learning system
- Robust fallback for API failures

Limitations

- LLM dependency introduces latency
- Simple rule-based fallback (can be improved)
- Drift detection is basic

4a — Architecture Diagram (8 Marks)

- The system integrates all previous tasks into a production ready pipeline.
- Architecture Diagram:



1. Data Ingestion Layer:

- Sources:
  - **Satellite imagery** – high-resolution images for large-area monitoring.
  - **Drone feeds** – UAV-captured images for localized, high-detail monitoring.
  - **IoT cameras** – ground-based sensors for real-time traffic or crop monitoring.
- Storage:
  - All raw data is stored in **cloud object storage**, e.g., AWS S3, Google Cloud Storage, or Azure Blob Storage.
  - Storage is organized with metadata tags for source, timestamp, and location.
- Preprocessing:
  - **Image tiling**: Each image is split into smaller tiles (512×512 pixels) to match model input dimensions.
  - **Normalization**: Pixel values are normalized to standard ranges to improve model inference consistency.
  - **Compression**: Optional lossless compression is applied to reduce storage costs and I/O overhead.

2. Inference Layer:

- This layer performs the core ML/AI processing:
- **Model Serving**:
  - **Traffic detection model**: Detects vehicles, pedestrians, and other road elements.
  - **Crop segmentation model**: Identifies crop areas, health status, and anomalies.
- **GPU Allocation**:
  - Dedicated GPUs (e.g., **A100** or **T4**) are used for high-throughput batch inference.
  - GPU scaling can be dynamic depending on workload.
- **Batching**:
  - Multiple tiles are processed in parallel to maximize GPU utilization.
  - Typical **batch size**: 16–32 tiles, optimized for latency vs. throughput trade-offs.

3. Agentic Orchestration Layer:

- Responsible for routing, decision-making, and self-optimization:
- **Agent Graph:**
  - **Orchestrator Agent**: Central routing agent that directs tiles to the correct model (traffic or crop).
  - **QC Agent**: Evaluates the quality of model outputs (confidence, detection score).
  - **Reporting Agent**: Prepares outputs for dashboards and external consumption.
  - **Memory Store**: Maintains a history of outputs for drift detection and performance tracking.

- **LLM Calls**:
  - Task routing (deciding which model pipeline to run).
  - Fallback strategies when confidence is low or anomalies occur.
  - Self-optimization suggestions (e.g., batch size adjustment).

- **Memory Store & Drift Detection**:
  - Historical predictions are stored and monitored.
  - Drift detection identifies model performance degradation over time.
  - Supports automated alerts or retraining triggers.

4. Output Layer:

- The system produces actionable outputs for end-users and downstream systems:
- **Formats**:
  - **GeoJSON**: For mapping detected objects or crop areas on GIS platforms.
  - **Heatmaps**: Visualize density of traffic or crop anomalies.
- **Dashboards**:
  - Real-time web dashboards showing performance metrics, tile processing status, and detected events.
- **Alerts**:
  - **Low-confidence predictions**.
  - **Significant drift or anomalies** detected in outputs.
  - Enables **human-in-the-loop intervention**

5. Feedback Loop:

- A critical component for continuous improvement:
  - **Metrics Captured**:
    - Confidence scores, mAP (mean Average Precision), detection rates.
  - **Training Dataset Curation**:
    - Low-confidence or failed predictions are flagged for human review and added to retraining datasets.
  - **Continuous Retraining**:
    - Models are retrained periodically or when drift thresholds are exceeded.
    - Ensures sustained performance across evolving data distributions.

4b — MLOps & Monitoring Plan (7 Marks)

1. Metrics to Montor in Production:

- To ensure model reliability, the system continuously monitors the following metrics:

    - **mAP (mean Average Precision)**: Measures detection and segmentation accuracy for traffic and crop models.
    - **Latency**: Time per tile to generate inference results. Critical for real-time applications (e.g., traffic monitoring).
    - **Throughput**: Number of tiles processed per unit time. Ensures the system can meet 10,000 tiles/day.
    - **Confidence Distribution**: Average confidence score of detections to flag low-confidence outputs.
    - **Drift Score**: Tracks data distribution and model performance drift over time.

2. Alerting Thresholds and Escalation:

- **Low-confidence predictions (<0.7)** → Trigger automated re-inference or human review.
- **Latency exceeds 1.5× baseline** → Alert DevOps team via Slack/Email.
- **mAP drop >5% compared to previous deployment** → Initiate retraining pipeline.
- **Significant drift detected** → Escalate to ML engineer for dataset review.
- Alerts are **logged and visualized** in monitoring dashboards (Grafana, W&B, or custom web dashboard).

3. Retraining Trigger Conditions:

- **Periodic retraining**: Weekly or monthly depending on new incoming data volume.
- **Performance-based retraining**: Triggered when drift or accuracy drop exceeds threshold.
- **Data versioning**: All retraining datasets are versioned and stored with timestamps, source info, and preprocessing parameters.

4. Model Registry and Versioning:

- **Model Registry**: MLflow or W&B used to track model versions, training datasets, hyperparameters, and deployment history.
- **Version Control**: Each model is tagged (e.g., traffic_v1.0, crop_v2.1) for reproducibility.
- **Deployment Strategy**: Canary deployments for new models with partial traffic, monitored closely before full rollout.

5. Logging and Observability:

- **Pipeline logs**: Input, output, processing time, and errors logged per tile.
- **Anomaly tracking**: Sudden drops in detection counts or extreme confidence values trigger alerts.
- **Human-in-the-loop**: For low-confidence or ambiguous outputs, flagged tiles are sent for review before inclusion in retraining datasets.

4c — Scalability & Cost Analysis (5 Marks)

1. **Estimated Inference Time per 512 X 512 Tile:**

GPU Type	Approx. Time per Tile	Notes
A100	30-50 ms	High throughput, suitable for Large scale production
T4	70-120 ms	Cost effective for moderate scale deployments

NOTE: Batch Size 16 to 32 recommended for optimal GPU Utilization.

2. Parallelization Strategy for 10000 tile/day:

- **Tiles per batch**: 32
- **Concurrent batches**: 8–16 (depending on GPU memory)
- **Processing nodes**: 2–4 A100 GPUs or 4–8 T4 GPUs
- **Estimated total time**:

  - A100 cluster: ~1–2 hours/day
  - T4 cluster: ~3–5 hours/day

NOTE: This ensures all tiles are processed within a day with room for retries.

3. Cloud Cost Estimate:

- **AWS** (as example):

  - 2 × A100 GPU instances (p4d.24xlarge) for batch inference → ~$25–30/hour
  - Storage (S3) for 10,000 tiles/day → ~$5–10/day
  - Network & additional services → ~$10/day
  - **Total**: ~$50/day for production-scale processing

- GCP/Azure prices are similar; T4 instances lower cost but longer runtime (~$20–25/day).

4. Edge Deployment Consideration:

- **Feasibility**:
  - T4 or smaller GPUs can be deployed on **drone-embedded hardware or edge devices**, but with reduced batch size.
  - For real-time drone use, only **critical inferences** (e.g., traffic hotspots or crop anomalies) should run on-edge; full batch processing remains cloud-based.
- **Trade-offs**:
  - Lower latency at the cost of reduced model complexity or smaller input resolution.
  - Continuous retraining and memory storage handled in the cloud.

❖ **CONCLUSION:** The system is scalable, cost-efficient, and can handle 10,000 tiles/day via GPU batch processing with cloud-based MLOps pipelines, while edge deployment is feasible for lightweight real-time tasks.